SOFTWARE SIZE AND REUSE ESTIMATING

SOFTWARE SIZE ESTIMATION

- Software size estimation involves predicting the size of the software product to be developed.

- **Different size measures**: (units of measurement for software)
  - Lines of code (LOC)
  - Function point
  - Feature points
  - Number of bubbles on a data flow diagram (DFD)
  - Number of entities on an entity relationship diagram (ERD)
  - Count of process/control (PSPEC/CSPEC) boxes on a structure chart
  - Number of “shalls” versus “wills” in a government specification
  - Amount of documentation
  - Number of objects, attributes, and services on an object diagram

BENEFITS

Accurate size estimation helps in :

1. resource planning
2. cost estimation
3. Scheduling
4. Overall project management.
5. Provides a baseline for tracking progress and assessing project performance.

TECHNIQUES OF COST ESTIMATION

- LOC (Lines of Code)
- Function Point Analysis
- Feature Point Analysis
- Object Points
- Model Blitz
- Wide Band Delphi

I. LINES OF CODE AS A UNIT OF SIZE

- **LOC (Lines of Code):**

  Estimates the size **based on the number of lines of code** expected to be written.

  Can be done using **historical data, expert judgment, or automated tools**.

  Measures source code size to predict project effort, cost, and duration.

1. LOC ESTIMATION BY EXPERTS OPINION

- **Decomposition:** Split the project scope into smaller, manageable software functions or modules.
- **Three-Point Sizing:** Estimate optimistic, pessimistic, and most likely LOC for each component to handle risk.
- **Expected Value Calculation:** Compute the expected LOC using the weighted average formula:

  (Optimistic + (4 * Most Likely)+ Pessimistic) / 6.
- **Effort and Cost Conversion:** Divide total estimated LOC by past team productivity rates (e.g., LOC per month) to find person-months and total project cost.

LINES OF CODE (LOC) ESTIMATION

- Planners break down tasks,

  estimate logical or physical lines for each function using historical data,

  apply productivity metrics (like LOC per person-month) to forecast delivery timelines.

2.ESTIMATING LOC BY ANALOGY

- Top-down project management technique
- Predicts the size of a new software system by comparing its characteristics to similar, previously completed historical projects with known LOC metrics
- Adjusting the final count based on structural differences and technical complexity

STEPS IN LOC ANALOGY ESTIMATION

- **Characterize the new project:**

  Identify key features like domain type, functionality, and architecture.
- **Search historical database:**

  Find past projects that match the functional profile closely.
- **Retrieve past LOC data:**

  Extract the exact recorded lines of code from those matched historical projects.
- **Apply adjustment factors:**

  Scale the historical LOC up or down using expert judgment for variances in scope, language differences, or target platform complexity.

GUIDELINES

- Ensure that each “source code line” counted contains only one source statement (if two executable statements appear on one line, separated by a semicolon, then the count is two)
- Count data definitions once.
- Count *all delivered, executable.*
- Do not count lines that contain only comments.
- Do not count debug code or other temporary code such as test software, test cases, development tools, prototyping tools, and so on.
- Count each invocation, call, or inclusion (don’t count reused source statements).
- Translate the number of lines of code to assembly language equivalent lines so that comparisons may be made across projects.

ADVANTAGES

- It is widely used and universally accepted.
- It permits comparison of size and productivity metrics
- It directly relates to the end product.
- LOC are easily measured upon project completion.
- It measures software from the developer's point of view.
- Continuous improvement activities exist for estimation techniques.

DISADVANTAGES

LOC is difficult to estimate for new software early in the life cycle.

- Vary with the type coding languages, with design methods, and with programmer style and ability.
- There are no industry standards (such as ISO) for counting lines of code.
  - LOC is more difficult than a “straight count.
- LOC cannot be used for normalizing.[if platforms or languages are different].
- Excess code that inflates or otherwise skews the LOC count.

FUNCTION POINTS AS A UNIT OF SIZE

FUNCTION POINTS

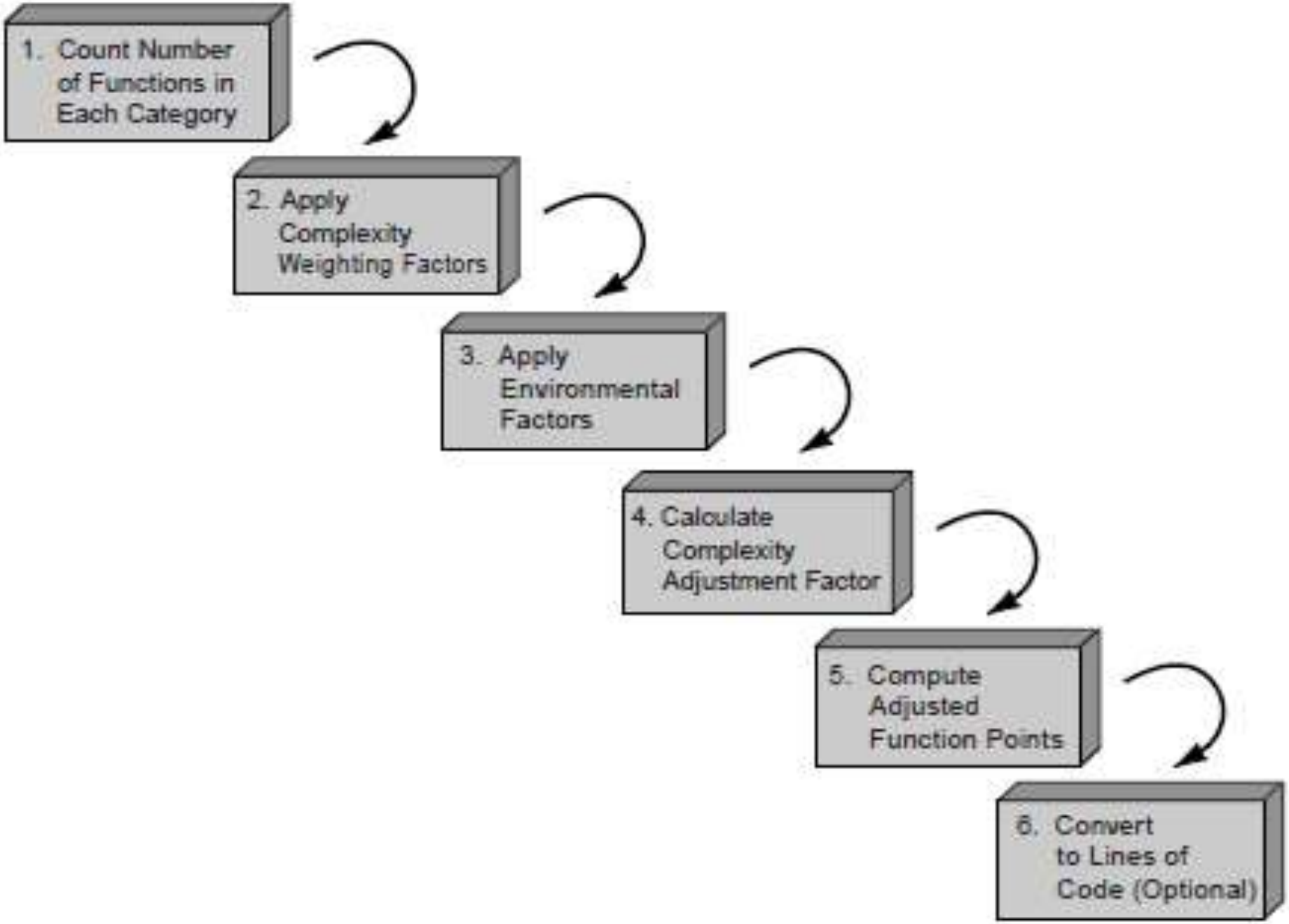
- **Function Point (FP)** is a software measurement technique used to measure the size of software based on the **functions it performs**, rather than simply counting the number of lines of code (LOC).
- Introduced in the late 1970s by **A. J. Albrecht of IBM** for transaction-oriented systems.
- Later, **Capers Jones** expanded the concept, and the **International Function Point User Group (IFPUG)** was formed in 1986

WHY FUNCTION POINTS ARE USED?

Function points are intended to:

- Measure categories of **end-user business functions.**
- Avoid depending on LOC estimates at an early stage of the software life cycle.
- Determine the number and complexity of:
  - Outputs
  - Inputs
  - Database inquiries
  - Files or data structures
  - External interface

GUIDELINES FOR / STEPS IN COUNTING FUNCTION POINTS


1. Count Number of Functions in Each Category
2. Apply Complexity Weighting Factors
3. Apply Environmental Factors
4. Calculate Complexity Adjustment Factor
5. Compute Adjusted Function Points
6. Convert to Lines of Code (Optional)

Step 1. Count Number of Functions in Each Category Step 2. Apply Complexity Weighting Factors				
	Simple	**Average**	**Complex**	**Function Points**
Number of Outputs	____ × 4	____ × 5	____ × 7	
Number of Inputs	____ × 3	____ × 4	____ × 6	
Number of Inquiry Outputs	____ × 4	____ × 5	____ × 7	
Number of Inquiry Inputs	____ × 3	____ × 4	____ × 6	
Number of Files	____ × 7	____ × 10	____ × 15	
Number of Interfaces	____ × 5	____ × 7	____ × 10	
			Total (FP):	

Function Point Analysis Worksheet Format

Step 3. Apply Environmental Factors	
Environmental Factor	**Rating (0, 1, 2, 3, 4, 5)**
Data Communications	
Distributed Computing	
Performance Requirements	
Constrained Configuration	
Transaction Rate	
Online Inquiry and/or Entry	
End-User Efficiency	
Online Update	
Complex Processing	
Reusability	
Ease of Conversion/Install	
Ease of Operation	
Used at Multiple Sites	
Potential for Function Change	
Total (N):	

Step 4. Calculate Complexity Adjustment Factor (CAF)

$CAF = 0.65 + (0.01 \times N) =$

Step 5. Compute Adjusted Function Points (AFP)

$AFP = FP\ (Raw) \times CAF =$

Step 6. Convert to LOC (Optional)

$LOC = AFP \times LOC/AFP =$

STEP 1. COUNT NUMBER OF FUNCTIONS IN EACH CATEGORY

category: (Outputs, Inputs, Database inquiries, Files or data structures, External interfaces)

General Guidelines for Counting

- Count only software requirements functions.
- Count logical representations.
- If two inputs or outputs require different processing logic, they are considered separate functions.
- The initial estimation involves examining major system components, such as:

[How much output is produced? input is required?
data is stored? inquiries are made?
files are involved?]

I)COUNTING OUTPUTS

- **External outputs** are information produced by the software and sent outside the system.
- Outputs represent **business information produced for the end user.**
- Examples:

  screen data, report data, error messages, and so on.

II)COUNTING INPUTS

Remember the following when counting inputs:

- External inputs are things received by the software from outside of the system.
- Inputs are units of business information input by the user to the software for processing or storage.
- Count each unique unit of input.

TABLE 10–3
Function Point Analysis Outputs Weighting Factors

	1–5 Data Items Referenced	6–19 Data Items Referenced	20 or More Data Items Referenced
0 or 1 File Referenced	Simple (4)	Simple (4)	Average (5)
2 or 3 Files Referenced	Simple (4)	Average (5)	Complex (7)
4 or More Files Referenced	Average (5)	Complex (7)	Complex (7)

TABLE 10–4
Function Point Analysis Inputs Weighting Factors

	1–5 Data Items Referenced	6–19 Data Items Referenced	20 or More Data Items Referenced
0 or 1 File Referenced	Simple (3)	Simple (3)	Average (4)
2 or 3 Files Referenced	Simple (3)	Average (4)	Complex (6)
4 or More Files Referenced	Average (4)	Complex (6)	Complex (6)

III) COUNTING INQUIRIES (OUTPUT/INPUT)

- **External Inquiry:** An online request/input from outside
- **Database Access:** Retrieves specific data using simple keys; **no data updates** are performed.
- **Count Each Unique Inquiry:**

  Its **input/output format is different**, or

  It has the same format but **different processing logic.**
- **Complexity:** Different input/output portions may have different **complexity weights.**
- **Queries ≠ Inquiries:** Queries are usually counted as **inputs or outputs.**

TABLE 10–5
Function Point Analysis Inquiries Weighting Factors

Output Part (Note: Select the Greatest Value—Output or Input)	1–5 Data Items Referenced	6–19 Data Items Referenced	20 or More Data Items Referenced
0 or 1 File Referenced	Simple (4)	Simple (4)	Average (5)
2 or 3 Files Referenced	Simple (4)	Average (5)	Complex (7)
4 or More Files Referenced	Average (5)	Complex (7)	Complex (7)
Input Part (Note: Select the Greatest Value—Output or Input)	**1–5 Data Items Referenced**	**6–19 Data Items Referenced**	**20 or More Data Items Referenced**
0 or 1 File Referenced	Simple (3)	Simple (3)	Average (4)
2 or 3 Files Referenced	Simple (3)	Average (4)	Complex (6)
4 or More Files Referenced	Average (4)	Complex (6)	Complex (6)

IV) COUNTING DATA STRUCTURES (FILES)

- Internal files are logical files within the program.
- Data structures (previously known as “files”) are each **primary logical group of user data** permanently stored entirely within the software system boundary.
- Data structures are available to users via inputs, outputs, inquiries, or interfaces.

V) COUNTING INTERFACES

- External files are machine-generated files used by the program.
- Interfaces are data (and control) stored outside the boundary of the software system being evaluated.
- Data structures shared between systems are counted as both interfaces and data structures.
- Count each data and control flow in each direction as a unique interface.

TABLE 10–6
Function Point Analysis Files Weighting Factors

Note: Count of Logical Relationships, Not Physical Record Types	1–9 Data Items Referenced	20–50 Data Items Referenced	51 or More Data Items Referenced
1 Logical Record Format/Relationship	Simple (7)	Simple (7)	Average (10)
2–5 Logical Record Format/Relationships	Simple (7)	Average (10)	Complex (15)
6 or More Logical Record Format/ Relationships	Average (10)	Complex (15)	Complex (15)

TABLE 10–7
Function Point Analysis Interfaces Weighting Factors

Note: Number of Logical Relationships, Not Physical Record Types	1–9 Data Items Referenced	20–50 Data Items Referenced	51 or More Data Items Referenced
1 Logical Record Format/Relationship	Simple (5)	Simple (5)	Average (7)
2–5 Logical Record Format/Relationships	Simple (5)	Average (7)	Complex (10)
6 or More Logical Record Format/ Relationships	Average (7)	Complex (10)	Complex (10)

STEP 2. APPLY COMPLEXITY WEIGHTING FACTORS

- Multiply each the number of each type (simple, average, complex) within each category

(output, input, inquiries [output/input], data structure [files], interfaces) by the appropriate weighting factor.

- Add the totals for each category.
- Notice that the total results in a “raw function point” count.

STEP 3. APPLY ENVIRONMENTAL FACTORS

- **Adjust the raw function point** total to account for **environmental factors** that affect the entire software development process.
- Many aspects of your surroundings might affect the software development process.
- Some of these **aspects affect the project positively**, and others tip the **scales in the negative direction**; all are considered as they uniquely apply to a specific project.
- **Rate each factor on a scale of 0 to 5** (where 0 means not applicable).
- Sum the factor ratings (Fn) to calculate a total environmental influence factor:

  N=sum(Fn)

STEP 4. CALCULATE COMPLEXITY ADJUSTMENT FACTOR (CAF)

- Barry Boehm postulate and Capers Jones supports the theory with empirical data:

  stating that **environmental factors would have a maximum impact of +/– 35% on the raw function point total.**

- CAF= 0.65+ (0.01 X N)

  where N is the sum of the weighted environmental factors.

STEP-3 (CONTINUED)

- Minimum CAF = 0.65 + (0.01 × 0) = 0.65.
  Maximum CAF = 0.65 + (0.01 × 70) = 1.35.

  Where N varies from 0 to 70, since we have 14 Environmental Factors. (1.35 – 0.65 = 0.70)

The earliest estimates of size and effort may be off by a factor +/– 35%.

STEP 5. COMPUTE ADJUSTED FUNCTION POINTS

Adjusted function points (AFP) =

raw function points × CAF

STEP 6. CONVERT TO LOC

Reason For Conversion

- To measure and compare the productivity or size of programs or systems that are written in multiple languages;
- To use the standard unit of measure for input into estimating tools
- To convert the size of a program or application in any language to the equivalent size if the application were written in a different language.

LOC = adjusted function points × LOC per adjusted function point

[AFP × # of LOC per AFP = LOC]

FUNCTION POINT ANALYSIS WORKSHEET EXAMPLE

Step 1. Count Number of Functions in Each Category
Step 2. Apply Complexity Weighting Factors

	Simple	Average	Complex	Function Points
Number of Outputs	12 × 4 = 48	11 × 5 = 55	5 × 7 = 35	48 + 55 + 35 = 138
Number of Inputs	8 × 3 = 24	9 × 4 = 36	6 × 6 = 36	24 + 36 + 36 = 96
Number of Inquiry Outputs	5 × 4 = 20	7 × 5 = 35	3 × 7 = 21	20 + 35 + 21
Number of Inquiry Inputs	5 × 3 = 15	8 × 4 = 32	4 × 6 = 24	15 + 32 + 24 = 71
Number of Files	12 × 7 = 84	3 × 10 = 30	2 × 15 = 30	84 + 30 + 30 = 144
Number of Interfaces	9 × 5 = 45	6 × 7 = 42	4 × 10 = 40	45 + 42+ 40 = 127
			Total (FP):	**652 "Raw" Function Points**

Step 3. Apply Environmental Factors	
Environmental Factor	**Rating (0, 1, 2, 3, 4, 5)**
Data Communications	5
Distributed Computing	5
Performance Requirements	3
Constrained Configuration	0
Transaction Rate	5
Online Inquiry and/or Entry	4
End-User Efficiency	5
Online Update	4
Complex Processing	2
Reusability	2

(Continues)

Environmental Factor	Rating (0, 1, 2, 3, 4, 5)
Ease of Conversion/Install	3
Ease of Operation	4
Used at Multiple Sites	5
Potential for Function Change	4
Total (N):	**51**

Step 4. Calculate Complexity Adjustment Factor (CAF)

$CAF = 0.65 + (0.01 \times N) = 0.65 + (0.01 \times 51) = 1.16$

Step 5. Compute Adjusted Function Points (AFP)

$AFP = FP\ (Raw) \times CAF = 652 \times 1.16 = 756.32$

Step 6. Convert to LOC (Optional)

LOC for the C Language = $AFP \times LOC/AFP = 756.32 \times 128 = 96{,}808.96$ LOC

SOFTWARE SIZE

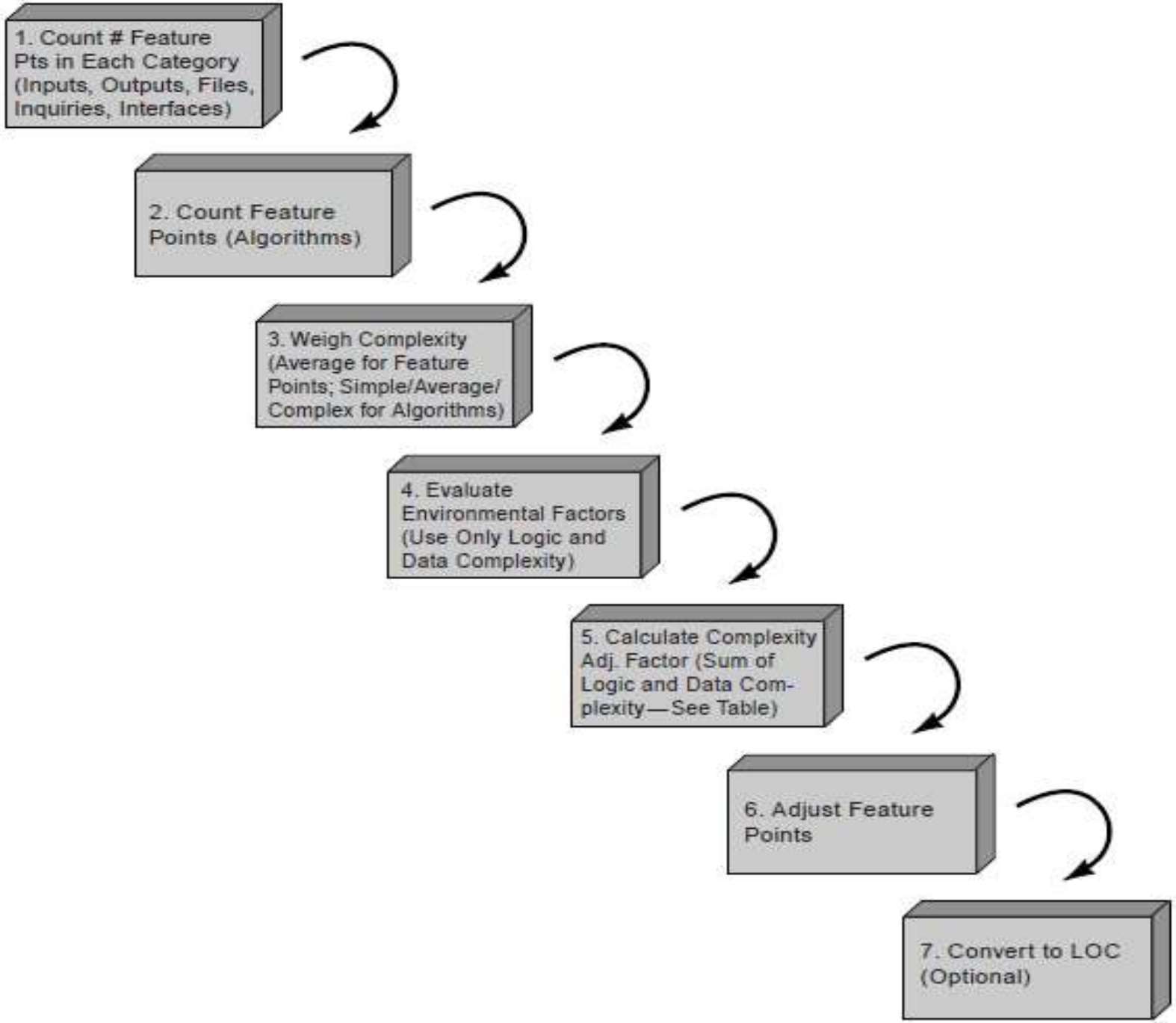
Feature Points

Feature Points as a Unit of Size

- An extension of the function point
- Deal with different kinds of applications, such as embedded and/or real-time systems.
- A feature point is a new category of function that represents complex algorithms and control (stimulus/response).
- The complexity of the algorithm is defined in terms of the number of “rules” required to express that algorithm.

[Definition: Feature points are basically function points that are sensitive to high algorithmic complexity, where an algorithm is a bounded set of rules (executable statements) required to solve a computational problem].

Guidelines for Counting Feature Points


1. Count # Feature Pts in Each Category (Inputs, Outputs, Files, Inquiries, Interfaces)
2. Count Feature Points (Algorithms)
3. Weigh Complexity (Average for Feature Points; Simple/Average/ Complex for Algorithms)
4. Evaluate Environmental Factors (Use Only Logic and Data Complexity)
5. Calculate Complexity Adj. Factor (Sum of Logic and Data Com- plexity—See Table)
6. Adjust Feature Points
7. Convert to LOC (Optional)

Step 1. Count Feature Points

- This is the same as counting function points—count inputs, outputs, files (data structures), inquiries, and interfaces.

Step 2. Continue the Feature Point Count by Counting the Number of Algorithms

- An algorithm is a bounded computational problem.
- deal with a definite, solvable, bounded problem with a single entry and a single exit point.
- data flow diagrams or structure charts in design equate an algorithm to a basic process specification or module specification.

Step 3. Weigh Complexity

- Use "average" weights

  instead of simple, average, or complex for inputs, outputs, files (data structures), inquiries, and interfaces.

- The default weighting factor for algorithms is 3. (can vary 1 to 10).
- Algorithms - basic arithmetic operations and few decision rules)- 1.
- Algorithms -Complex equations, matrix operation, and difficult logical Processing - 10.

- Algorithms should be counted - have these characteristics:

  Deals with a solvable, bounded, definite problem;

  Must be finite and have an end;

  Is precise and unambiguous;

  Has output or produces a result;

  Has an input or starting value;

  Is implementable;

  Is capable of representation via sequence, if then-else, do-case, do-while, and do-until.

Step 4. Evaluate Environmental Factors

- Instead of the 14 environmental factors used in function point analysis, feature point uses only two:

  logic complexity and data complexity. (range is from 1 to 5).

Logic Values

1—Simple algorithms and calculations
2—Majority of simple algorithms
3—Average complexity of algorithms
4—Some difficult algorithms
5—Many difficult algorithms

Data Values

1—Simple data
2—Numerous variables, but simple relationships
3—Multiple fields, files, and interactions
4—Complex file structures
5—Very complex files and data relationships

- Sum the logic and data complexity factor values, yielding a number between 2 and 10.

Step 5. Calculate the Complexity Adjustment Factor

TABLE 10–12
Feature Point Complexity Adjustment Factor

Sum of Logic and Data Complexity	Complexity Adjustment Factor
2	0.6
3	0.7
4	0.8
5	0.9
6	1.0
7	1.1
8	1.2
9	1.3
10	1.4

- **Step 6. Multiply the Raw Feature Point Count by the Complexity Adjustment Factor**

- **Step 7. Convert to Lines of Code Using the Function Point Translation Table(Optional)**

Example:

TABLE 10–13
Feature Point Analysis Worksheet Example

Step 1. Count Feature Points		
	Average	**Feature Points**
Number of Inputs	12 × 4	48
Number of Outputs	15 × 5	75
Number of Files (Data Structures)	22 × 7	154
Number of Inquiries	17 × 4	68
Number of Interfaces	8 × 7	56

(Continues)

TABLE 10–13 (Continued)
Feature Point Analysis Worksheet Example

Step 2. Count the Number of Algorithms		
	Average	**Feature Points**
Number of Algorithms	43 × 3	129
Total (FP):	530 "Raw" Feature Points	

Step 3. Weigh Complexity **Step 4. Evaluate Environmental Factors** **Step 5. Calculate the Complexity Adjustment Factor (CAF)**
Logic Values (Select One) Simple Algorithms and Calculations—1 Majority of Simple Algorithms—2 Average Complexity of Algorithms—3 →**Some Difficult Algorithms—4** Many Difficult Algorithms—5
Data Values (Select One) Simple Data—1 Numerous Variables, but Simple Relationships—2 →**Multiple Fields, Files, and Interactions—3** Complex File Structures—4 Very Complex Files and Data Relationships—5
Total (CAF): 4 + 3 = 7 Complexity Adjustment Factor

Step 6. Multiply the Raw Feature Point Count by the CAF

Raw FP × CAF = 530 × 7 = 3,710 Adjusted Feature Points

Step 7. Convert to Lines of Code (Optional)

LOC for the Java Language = AFP × LOC/AFP = 3,710 × 53 = 196,630 LOC

- **Advantages**

  excellent approach to use in the size estimation of algorithmically intensive systems.

- **Disadvantages**

  subjective classification of algorithmic complexity.

SOFTWARE SIZE

Object Points

Object Points

- Counting “object points” to determine software size is an approach

  developed for object-oriented technology.
- Conducted at a more macro level than function points
- Assigns one object point to each unique class or object, such as a screen, output, report, and so on.
- The rest of the process is similar to that of function and feature points, but the conversion factors are different.

SOFTWARE SIZE

Model Blitz

- The concept of blitz modeling is based on **Tom DeMarco's bang metric**.
- Counting component pieces of the system (**design elements**) and multiplying the count by a **productivity factor**
- **For example**, high-level data flow diagrams or object models:

  Imagine that there are **20 object classes** are implemented on average as **5 procedural programs** per class. The average size procedural program (C language) is **75 LOC**.

  Then the size can quickly be calculated as: [This is known as a "blitz" of early feasibility documents]

Number of processes (object classes) × Number of programs per class × Average Program Size
= Estimated Size

20 object classes × 5 programs per class × 75 LOC per program = 7,500 LOC estimated

- **Another Example**

: If it is known that each process bubble on a Level 0 DFD(having **7 major process bubbles)** roughly corresponds **to 4 actual SQL language programs**, and it is also known that the average size for programs in the **SQL library is 350 LOC**, then a simple multiplication will suffice for the initial size count.

Number of processes (DFD bubbles) × Number of programs per bubble × Average Program Size = Estimated Size

7 bubbles × 4 programs per bubble × 350 LOC per program = 9,800 LOC estimated

- If a high-level global object model is produced during the feasibility scoping phase, and it is ***known from historical evidence***

  :that each service corresponds to **2 C++ programs** and that company standards encourage confining each service packet to **100 LOC or less**, then multiplying as follows will provide a good start in the estimation of the size of the system in LOC:

  Number of services × 2 × 100

- When actual component sizes and their corresponding amounts of development effort are known, then average “productivity” is also known

  (size ÷ effort).

- **DeMarco suggests, with the bang metric**, that function-strong systems (e.g., real time) be computed separately from data-strong systems.
- **Functions-strong systems** rely on a count of indivisible functional primitives as defined by a data flow diagram.
- **Data-strong systems** rely on a count of objects in the system-global data model. Each will have a **weighting factor (WF)** applied.
- **An example (**with function-strong systems):
- WF (average number of modules needed to complete this function) is 3, the number of processes plus control specifications (functions) is 8, and the average size per function is 78 LOC. Then:

$$\text{WF} \times (\text{Number of process and control specifications}) \times \text{average LOC for this type of module} = \text{LOC}$$

$$3 \text{ modules needed for function} \times 8 \text{ functions} \times 78 \text{ LOC} = 1{,}872 \text{ LOC}$$

Advantages of Model Blitz

- It is easy to use with structured methods and with object-oriented classes, services, and so on.
- Accuracy increases with use of historical data.
- Estimated size can be easily compared with the actual size during post-project analysis.

Disadvantages of Model Blitz

- It requires use of design methodology.
- Estimation cannot begin until design is complete.
- It requires historical data.
- It does not evaluate environmental factors.

Software size Estimation

Wideband Delphi

Delphi Technique

- The **Delphi technique** originated at the Rand Corporation decades ago.
- This is a disciplined method of **using the experience of several people** to reach an estimate that incorporates **all of their knowledge**.
- The “pure” approach is to **collect** expert opinion **in isolation**, feed back anonymous summary results, and iterate (**without group discussion**).

Guidelines for Conducting Wideband Delphi Group Consensus

Because the Delphi approach take a very long time, the concept of **Wideband Delphi** was introduced to **speed up the process**. This improved approach **uses group discussion**.

Steps in Conducting Wideband Delphi

There are **six major steps** in conducting Wideband Delphi:

1. Present experts with a problem and a response form.

2. Conduct a group discussion.

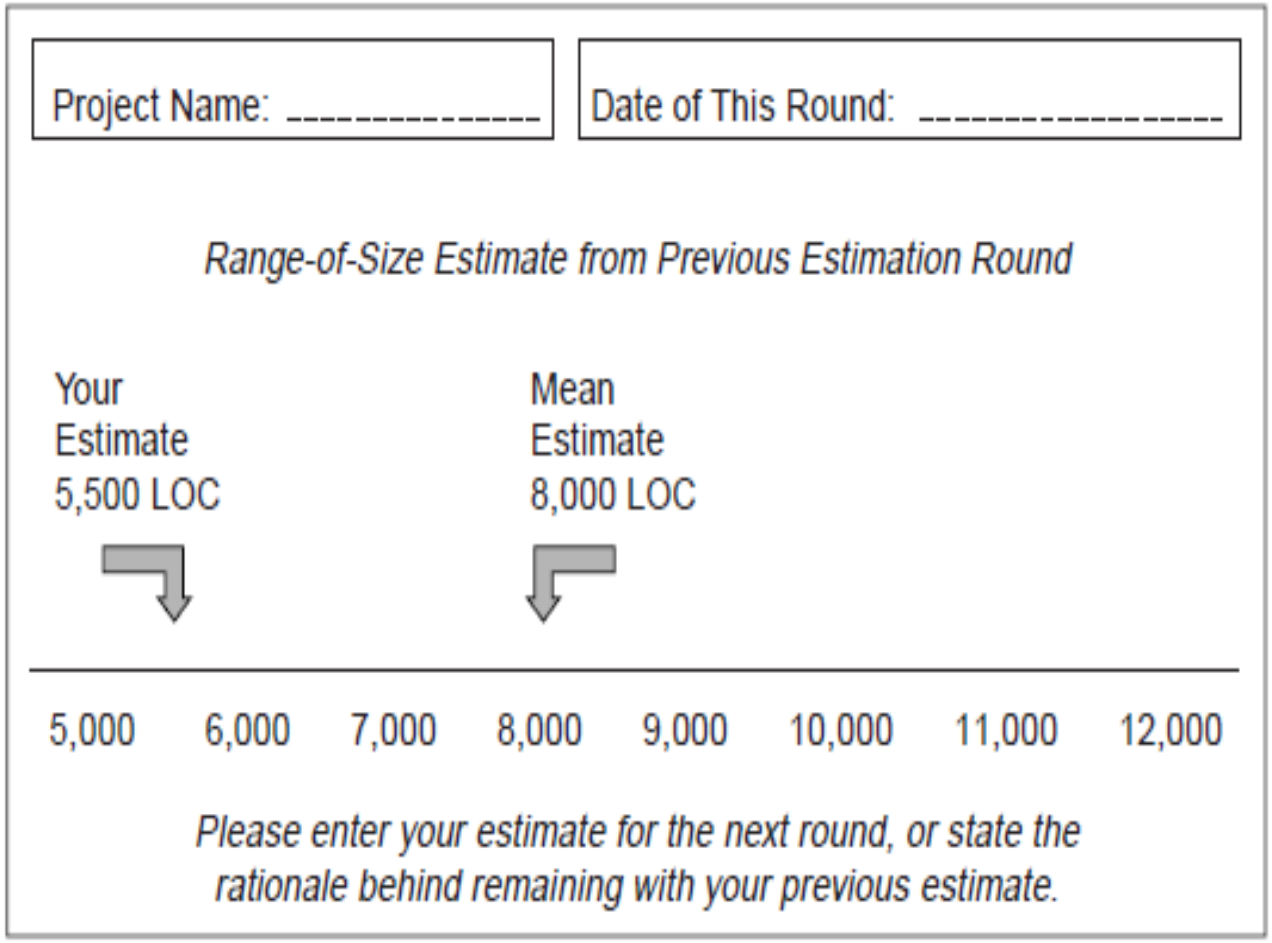
3. Collect expert opinion anonymously.

4. Feed back a summary of results to each expert.

5. Conduct another group discussion.

6. Iterate as necessary until consensus is reached.

Result Summary Form



Advantages of Wideband Delphi

- **Implementation is easy and inexpensive**.
- It **does not require historical data**, although it is useful if available.
- Results are **more accurate and less "dangerous"** than LOC estimating.
- It aids in providing a **global view of project** to team members.

Disadvantages of Wideband Delphi

- It is difficult to repeat **with a different group of experts**.
- You may fail to reach consensus r it may reach on an **incorrect estimate**
- You can develop a false sense of confidence.
- Experts may be **all biased** in the same subjective direction.

The Effects of Reuse on Software Size
(or)
Reuse Estimation

Reuse terminology

- **New code**—code developed for a new application **without including large portions of previously written code**
- **Modified code** —code developed for a previous application that is suitable for a new application after a ***modest amount of modification***
- **Reused code** —code developed for a previous application that is suitable for a new application ***without change of any kind***
- **Legacy code** —code developed for a previous application that is ***believed to be of use*** for a new application

Estimation

- **The first step** : Separate new code from modified and reused code.
- This is because modified and reused code can **almost never just be added in** —there are changes required for the integration, and size and effort increases to accommodate those changes.

Example

TABLE 10–14
Separate New, Modified, and Reused Lines of Code

Item	New LOC	Modified LOC	Reused LOC	Total LOC
Component 1	1,233	0	0	1,233
Component 2	0	988	0	988
Component 3	0	0	781	781
Component 4	560	245	0	805
Component 5	345	549	420	1,314
Total	2,138	1,782	1,201	5,121

- **A rule of thumb is to examine** the smallest level known for a unit or module (typically about 100 LOC).
- If the unit :

  No change at all -> "reused."

  Change : one comment or executable statement -> "modified."

  Change : more than 50% -> "new."

In addition, once the modified code has been identified, separate it into categories : (representing the type of modification)

- **correcting defects** from the modifications for **adding enhancements.**

Separation of Modified Code

TABLE 10–15
Different Kinds of Modified Code

Item	New LOC	Modified to Fix Bugs	Modified to Add Enhancements	Reused LOC	Total LOC
Component 1	1,233	0	0	0	1,233
Component 2	0	988	0	0	988
Component 3	0	0	0	781	781
Component 4	560	0	245	0	805
Component 5	345	302	247	420	1,314
Total Delivered LOC	2,138	1,782	1,782	1,201	5,121

Applying Reuse Factor

- Reuse factors come from experience.
- The 30% and 60% **factors** were observed on hundreds of projects.
- The best indicator of size and effort is from actual data — **keeping track of estimates and actuals in a historical database**.
- Applying reuse factors indicating :

  Reused software takes 30% as much effort as new software

  modified software takes about 60% as much effort as new software.

  Hence , the total effort to develop **5,121 LOC will be comparable to that of producing 3,567 lines of new code**.

Result of Reuse Factor

TABLE 10–16
Applying Reuse Factors

Item	New LOC	Modified LOC	Reused LOC	Total LOC
Total	2,138	1,782	1,201	5,121
Factor	100%	60%	30%	—
Net	2,138	1,069	360	3,567

TABLE 10–17
Typical Reuse Factors

Ease of Use	Reused	Modified
Easy	10%	25%
Average	30%	60%
Hard	40%	75%

More Accurate with Reuse

- Suppose an organization spends:
  18% of its time -> requirements
  25% -> design
  25% in code and test,
  32% in integration

For Modified :

1782(18%*20% +25%*40%+25%*70%+32%*100%)
=1782(3.6%+10%+17.5% +32%)
=1782(63.1%)
=1124

TABLE 10–18

Applying an Accurate Estimating Method to Reused and Modified Code

	Process Step	Requirements	Design	Code	Integration	
	Percent	18%	25%	25%	32%	
Delivered						Equivalent
2,138	New	100%	100%	100%	100%	2,138
1,782	Modified	20%	40%	70%	100%	1,124
1,201	Reused	10%	0%	0%	100%	406
5,121						3,668